

VS Code Debugging Guide for Ray Tracing

Setup Instructions

1. Create VS Code Configuration Files

Create these folders and files in your project directory:

```
your_project/
├── .vscode/
│   ├── launch.json
│   ├── tasks.json
│   └── c_cpp_properties.json
├── main_debug.cpp
├── classes.hpp
├── intersection.cpp
├── bitmap_image.hpp
└── scene.txt
```

2. Build and Debug Commands

Option A: Use VS Code Tasks (Recommended)

1. Press `Ctrl+Shift+P` and type "Tasks: Run Task"
2. Select "build debug"
3. Then press `F5` to start debugging

Option B: Manual Command Line

```
powershell
```

```
# Build debug version
```

```
g++ -std=c++11 -Wall -Wextra -g3 -O0 -DDEBUG -o raytracer_debug.exe main_debug.cpp -lopengl32 -lglu32 -lfreeglut
```

```
# Run in debug mode
```

```
gdb raytracer_debug.exe
```

3. Debug Features Added

Debug Macros

- `DEBUG_PRINT(x)` - Prints debug message with [DEBUG] prefix

- `DEBUG_VECTOR(name, vec)` - Prints vector components
- `DEBUG_CAMERA()` - Prints complete camera state

Function Call Tracking

- Every function call is numbered and logged
- Before/after states are printed for rotations
- Extensive logging in key areas

Breakpoint Locations

I've marked key breakpoint locations with comments like `// BREAKPOINT HERE`

How to Debug Step by Step

Method 1: VS Code Integrated Debugging

1. **Open VS Code** in your project folder
2. **Set breakpoints** by clicking in the gutter next to line numbers
3. **Press F5** to start debugging
4. **Use debug controls:**
 - F5: Continue
 - F10: Step Over
 - F11: Step Into
 - Shift+F11: Step Out
 - Ctrl+Shift+F5: Restart

Method 2: Console Debugging

1. **Compile with debug flags:**

powershell

```
g++ -std=c++11 -g3 -O0 -DDEBUG -o raytracer_debug.exe main_debug.cpp -lopengl32 -lglu32 -lfreeglut -lm
```

2. **Run and watch console output:**

powershell

```
.\raytracer_debug.exe
```

3. **Press keys 1/2** and watch the detailed output

Key Breakpoint Locations

1. Rotation Functions

cpp

// In keyboardListener - when '1' or '2' is pressed

case '1':

rotateLeft(); *// BREAKPOINT HERE*

// In rotateLeft function

void rotateLeft() {

... *// BREAKPOINT HERE - start of function*

... Vector3D new_l = l * cos(angle) + r * sin(angle); *// BREAKPOINT HERE - calculation*

l = new_l.normalize(); *// BREAKPOINT HERE - assignment*

}

2. Camera Setup

cpp

void calculateCamera() {

... *// BREAKPOINT HERE - camera vector calculation*

... l = look.normalize();

... r = l.cross(up).normalize();

... u = r.cross(l).normalize();

}

3. OpenGL Setup

cpp

void init() {

... calculateCamera(); *// BREAKPOINT HERE*

loadData(); *// BREAKPOINT HERE*

}

4. Ray Tracing

cpp

```
void capture() {  
    ....// BREAKPOINT HERE - start of ray tracing  
    for (int i = 0; i < imageWidth; i++) {  
        .... Ray ray(eye, rayDir); // BREAKPOINT HERE  
        .... double t = objects[k]->intersect(&ray, dummy_color, 0); // BREAKPOINT HERE  
    }  
}
```

Debugging the Rotation Issue

Step-by-Step Debugging Process:

1. **Set breakpoint in** `keyboardListener` when key '1' is pressed
2. **Step into** `rotateLeft()` function
3. **Watch the variables:**
 - `l` (look vector) before rotation
 - `r` (right vector) before rotation
 - `angle` value
 - `new_l` and `new_r` after calculation
 - `l` and `r` after normalization
4. **Verify vector changes:**
 - Look direction should change
 - Vectors should remain unit length (≈ 1.0)
 - Right vector should be perpendicular to look

Watch Window Variables

Add these to your VS Code watch window:

```
l.x, l.y, l.z  
r.x, r.y, r.z  
u.x, u.y, u.z  
eye.x, eye.y, eye.z  
angle  
function_call_count
```

Expected Behavior

When you press '1' (rotate left):

1. Console shows: "Key pressed: 49 ('1')"
2. Console shows: "Rotating left..."
3. Look direction values should change
4. OpenGL view should rotate

Debug Output Example

When working correctly, you should see:

```
[DEBUG] === rotateLeft() START (call #1) ===  
[DEBUG] BEFORE rotation:  
[DEBUG] === CAMERA STATE ===  
[DEBUG] Eye: (100, 100, 50)  
[DEBUG] Look: (-0.707107, -0.707107, 0)  
[DEBUG] Up: (0.5, -0.5, 0.707107)  
[DEBUG] Right: (0.5, -0.5, -0.707107)  
[DEBUG] =====  
[DEBUG] Rotation angle: 0.1 radians (5.72958 degrees)  
[DEBUG] Calculated new vectors:  
[DEBUG] new_l: (-0.657059, -0.75368, 0)  
[DEBUG] new_r: (0.45464, -0.547446, -0.707107)  
[DEBUG] AFTER rotation and normalization:  
[DEBUG] === CAMERA STATE ===  
[DEBUG] Eye: (100, 100, 50)  
[DEBUG] Look: (-0.657059, -0.75368, 0)  
[DEBUG] Up: (0.547446, -0.45464, 0.707107)  
[DEBUG] Right: (0.533277, -0.621548, -0.574025)  
[DEBUG] =====  
Look direction: (-0.657059, -0.75368, 0)  
[DEBUG] === rotateLeft() END ===
```

Troubleshooting

If rotation still doesn't work:

1. Check if `keyboardListener` is called:
 - Should see "Key pressed: 49 ('1')"
 - If not, keyboard callback isn't set up properly
2. Check if `rotateLeft()` is called:

- Should see "Rotating left..." in console
- If not, switch statement isn't working

3. Check vector calculations:

- Look direction should change numerically
- If values don't change, math is wrong

4. Check OpenGL update:

- `glutPostRedisplay()` should be called
- If display doesn't update, OpenGL isn't refreshing

Common Issues:

- **No console output:** Debug build not working
- **Function not called:** Keyboard callback issue
- **Values don't change:** Math error in rotation
- **View doesn't update:** OpenGL display issue

Quick Test Commands

powershell

Test compilation

```
g++ -std=c++11 -g3 -O0 -DDEBUG -o test.exe main_debug.cpp -lopengl32 -lglu32 -lfreeglut -lm
```

Check if debug macros work

```
echo 'Press 1 or 2 and check console output'
```

Test rotation math separately

```
g++ -o test_rotation.exe test_rotation.cpp
```

```
.\test_rotation.exe
```

This debug setup should help you identify exactly where the rotation issue is occurring!